The lux Hub Replicator: a Z Specification

Formal model of the single-writer, two-lock replication discipline

July 2026

1 Introduction

An MCP `clear` call once froze an agent for about 38 minutes: the tool sent to the display synchronously, on the agent's own thread, over a socket with no send timeout, while holding the client send lock. The fix moves every send to the display onto one background worker inside the Hub — the *replicator* — and keeps every agent-facing tool on the Hub's in-memory store, which it updates and returns from at once.

The worker introduces real concurrency. It is a background thread; it drains a set of changed scenes that several other threads mark; it reads a store that those threads mutate; and it kills and restarts a wedged display while those threads keep running. This document models that concurrency as a finite state machine over a small fixed set of scenes and mutator threads, so that ProB can enumerate every interleaving and either confirm the invariants or produce the exact offending trace.

Five properties must hold across every interleaving:

I1 — single writer. Only the worker ever writes to the display connection. No mutator thread holds the client send lock.

I2 — agent-path liveness. No mutator operation is ever disabled by display I/O or by the client send lock. A mutator makes progress even while the worker is stuck sending to a dead display.

I3 — no lock cycle. No thread ever holds the store lock and the client send lock at the same time. The worker snapshots under the store read lock, releases it, and only then takes the client lock to send.

I4 — no lost update. Whenever the system comes to rest with the display up, the display shows the store's latest state for every scene — including after a reap and respawn, which re-marks every live scene.

I5 — one display. The worker never spawns a fresh display while one is still alive; a respawn follows a kill.

2 Basic Types

$[SCENE, MUTATOR]$

`SCENE` is the set of scenes the Hub can hold; two suffice to exhibit a coalesced batch and a per-scene lost update. `MUTATOR` is the set of threads that change the store: MCP tool calls (`show`, `update`, `clear`) and Hub-side click handlers. Both kinds do the same thing to the store, so one carrier covers both; two mutators exhibit lock contention. Both carriers are bounded by ProB's `DEFAULT_SETSIZE`.

3 Free Types

$VAL ::= vempty \mid vlive \mid vtorn$

The content of one scene in the store or on the display, abstracted to a token. `vempty`: no elements (a scene that has been cleared or never populated). `vlive`: a committed, consistent scene tree. `vtorn`: the half-updated state a scene passes through while a mutator is inside a multi-step `replace_scene` — the value that, if read without the lock, is the "dictionary changed size during iteration" defect. Real scene structure is elided; what matters is whether a read observes a committed value or a torn one.

$$DPROC ::= dup \mid dstuck \mid ddead$$

The display process as the worker's send sees it. `dup`: alive and draining its socket; a send succeeds. `dstuck`: alive but no longer reading, so the send buffer fills and the time-limited send raises `BlockingIOError`. `ddead`: the peer is gone, so the send raises a plain `OSError` (`ECONNRESET`/`EPIPE`).

$$LOCK ::= held \mid free$$

$$FLAG ::= set \mid clear$$

Two-valued markers for the worker's read lock and client lock, and for the cleared and shutting signals, respectively. A dedicated free type is used rather than a built-in boolean so the spec is self-contained for `fuzz`.

$$\begin{aligned} WPHASE ::= \\ & widle \mid wdrain \mid wproc \mid wsnap \mid wsend \mid \\ & wreap \mid wensure \mid wremark \mid wreconn \mid \\ & wflush \mid wstopped \end{aligned}$$

The worker's own program counter. `widle`: waiting on the condition. `wdrain`: woken, about to take the whole changed-scene set. `wproc`: iterating the drained batch. `wsnap`: holding the store read lock, copying one scene's roots. `wsend`: holding the client lock, sending that copy. `wreap`/`wensure`/`wremark`: the reap-respawn-remark recovery after a send timeout. `wreconn`: the reconnect recovery after a dead peer. `wflush`/`wstopped`: the final bounded flush at shutdown.

4 State

The state is flat: one schema. Its predicate carries only *structural* coherence — the locks and the snapshot hold at most one thing, and the write lock is held exactly when a scene is mid-replace. These hold in every design, correct or buggy, so they constrain the state space without masking the safety properties. The safety properties (Section 10) are deliberately *absent* here, so that a violating state stays reachable and ProB can find it.

Repl

$$\begin{aligned} & store : SCENE \longrightarrow VAL \\ & displayed : SCENE \longrightarrow VAL \\ & dirty : \mathbb{P}\, SCENE \\ & batch : \mathbb{P}\, SCENE \\ & cleared : FLAG \\ & snapScene : \mathbb{P}\, SCENE \\ & snapVal : VAL \\ & storeLockMut : \mathbb{P}\, MUTATOR \\ & mutScene : \mathbb{P}\, SCENE \\ & readLock : LOCK \\ & clientLock : LOCK \\ & display : DPROC \\ & shutting : FLAG \\ & wphase : WPHASE \\ \hline & \#storeLockMut \leq 1 \\ & \#snapScene \leq 1 \\ & \#mutScene \leq 1 \\ & (storeLockMut = \varnothing \Leftrightarrow mutScene = \varnothing) \end{aligned}$$

`store` is the Hub's authoritative copy; `displayed` is what the display is currently showing. `dirty` is the changed-scene set guarded by the worker's condition, and `cleared` the cleared flag beside it. `batch` is the set the worker drained and is now pushing. `snapScene`/`snapVal` are the one scene the worker has copied out under the read lock and the value it copied. `storeLockMut` names the mutator holding the store write lock (empty or one), and `mutScene` the scene it is mid-replacing. `readLock` and `clientLock`

record whether the worker holds the store read lock and the client send lock. `display` is the display process state; `shutting` the clean-shutdown signal; `wphase` the worker's program counter.

Modelling the store read/write lock as *one* mutually-exclusive slot (`readLock` for the sole reader, `storeLockMut` for the one writer) is sound because there is exactly one reader thread in this design — the worker. A read/write lock exists to allow concurrent readers; with a single reader the only exclusion that can ever fire is reader-versus-writer, which a single slot captures.

5 Initialization

Init

$$
\begin{array}{l}
Repl' \\
\hline
store' = (\lambda\, s : SCENE \bullet vempty) \\
displayed' = (\lambda\, s : SCENE \bullet vempty) \\
dirty' = \varnothing \\
batch' = \varnothing \\
cleared' = clear \\
snapScene' = \varnothing \\
snapVal' = vempty \\
storeLockMut' = \varnothing \\
mutScene' = \varnothing \\
readLock' = free \\
clientLock' = free \\
display' = dup \\
shutting' = clear \\
wphase' = widle
\end{array}
$$

A single initial state: empty store, empty display, nothing dirty, no locks held, the worker idle, the display up.

6 Mutator Operations

A mutator changes the store under the write lock in two steps, so the scene passes through a torn intermediate the worker must never read. The first step takes the lock and begins the replace; the second commits the new value, marks the scene dirty, and releases the lock. Marking dirty is part of the commit: the tool calls `mark_dirty` on the same thread, immediately after `replace_scene` returns and before it yields, so no other thread observes the store between the store write and the mark.

The write lock is taken only when it is free *and* the worker is not mid-snapshot (`readLock = free`). That guard is the store lock's reader-writer exclusion; removing it from the worker's snapshot yields the buggy variant of Section 12.

ReplaceBegin

$\Delta Repl$
$m? : MUTATOR$
$s? : SCENE$

$storeLockMut = \varnothing$
$readLock = free$
$storeLockMut' = \{m?\}$
$mutScene' = \{s?\}$
$store' = store \oplus \{s? \mapsto vtorn\}$
$displayed' = displayed$
$dirty' = dirty$
$batch' = batch$
$cleared' = cleared$
$snapScene' = snapScene$
$snapVal' = snapVal$
$readLock' = readLock$
$clientLock' = clientLock$
$display' = display$
$shutting' = shutting$
$wphase' = wphase$

ReplaceCommit

$\Delta Repl$
$m? : MUTATOR$

$m? \in storeLockMut$
$store' = store \oplus (mutScene \times \{vlive\})$
$dirty' = dirty \cup mutScene$
$storeLockMut' = \varnothing$
$mutScene' = \varnothing$
$displayed' = displayed$
$batch' = batch$
$cleared' = cleared$
$snapScene' = snapScene$
$snapVal' = snapVal$
$readLock' = readLock$
$clientLock' = clientLock$
$display' = display$
$shutting' = shutting$
$wphase' = wphase$

The `clear` tool empties the whole store under the write lock and sets the cleared flag instead of marking a scene dirty. Modelled as one step, since the torn-read race is already exhibited by `replace`:

ClearStore

$\Delta Repl$

$storeLockMut = \varnothing$
$readLock = free$
$store' = (\lambda\, s : SCENE \bullet vempty)$
$cleared' = set$
$displayed' = displayed$
$dirty' = dirty$
$batch' = batch$
$snapScene' = snapScene$
$snapVal' = snapVal$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$readLock' = readLock$
$clientLock' = clientLock$
$display' = display$
$shutting' = shutting$
$wphase' = wphase$

7 The Worker Loop

The worker waits on the condition, wakes when something is dirty or the screen was cleared, drains the whole set, and pushes each scene by copying its current roots under the read lock and sending the copy outside the lock. Coalescing needs no extra mechanism: many marks of one scene add it to the set once, and the worker reads that scene's current store value when it snapshots, so the display receives the newest value, never a half-finished one.

Wake

$\Delta Repl$

$wphase = widle$
$shutting = clear$
$(dirty \neq \varnothing \lor cleared = set)$
$wphase' = wdrain$
$store' = store$
$displayed' = displayed$
$dirty' = dirty$
$batch' = batch$
$cleared' = cleared$
$snapScene' = snapScene$
$snapVal' = snapVal$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$readLock' = readLock$
$clientLock' = clientLock$
$display' = display$
$shutting' = shutting$

`Drain` takes the whole changed-scene set at once and clears it, holding the cleared flag for the end of the cycle. New marks that arrive after this point re-populate `dirty` for the next cycle.

Drain

$\Delta Repl$

$wphase = wdrain$
$batch' = dirty$
$dirty' = \varnothing$
$wphase' = wproc$
$store' = store$
$displayed' = displayed$
$cleared' = cleared$
$snapScene' = snapScene$
$snapVal' = snapVal$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$readLock' = readLock$
$clientLock' = clientLock$
$display' = display$
$shutting' = shutting$

`Snap` takes the store read lock, picks one scene still in the batch, and copies its current value. The read lock is available only when no mutator holds the write lock (`storeLockMut = ∅`); that guard is what stops the worker reading a torn scene. The guard `cleared = clear` holds a scene's paint until any pending clear has been applied, so a cleared cycle blanks before it repaints.

Snap

$\Delta Repl$

$wphase = wproc$
$batch \neq \varnothing$
$cleared = clear$
$storeLockMut = \varnothing$
$(\exists\, s : SCENE \bullet$
$\quad s \in batch \land snapScene' = \{s\} \land snapVal' = store\, s)$
$readLock' = held$
$wphase' = wsnap$
$store' = store$
$displayed' = displayed$
$dirty' = dirty$
$batch' = batch$
$cleared' = cleared$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$clientLock' = clientLock$
$display' = display$
$shutting' = shutting$

`SendBegin` releases the store read lock and only then takes the client send lock. This release-before-acquire is the whole of I3: the two locks are never held together, so no cycle can form.

SendBegin

$\Delta Repl$

$wphase = wsnap$
$readLock' = free$
$clientLock' = held$
$wphase' = wsend$
$store' = store$
$displayed' = displayed$
$dirty' = dirty$
$batch' = batch$
$cleared' = cleared$
$snapScene' = snapScene$
$snapVal' = snapVal$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$display' = display$
$shutting' = shutting$

The time-limited send has three outcomes, keyed on the display process. On success the display now shows the copied value and the scene leaves the batch.

SendOk

$\Delta Repl$

$wphase = wsend$
$display = dup$
$clientLock' = free$
$displayed' = displayed \oplus (snapScene \times \{snapVal\})$
$batch' = batch \setminus snapScene$
$snapScene' = \varnothing$
$snapVal' = vempty$
$wphase' = wproc$
$store' = store$
$dirty' = dirty$
$cleared' = cleared$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$readLock' = readLock$
$display' = display$
$shutting' = shutting$

On a send timeout (`BlockingIOError`) the display is alive but wedged. The send gives up within its time limit and releases the client lock, and the worker turns to the reap-respawn path. The batch is kept: the respawn re-marks every live scene, which re-covers the scene in flight.

SendStuck

$\Delta Repl$

$wphase = wsend$
$display = dstuck$
$clientLock' = free$
$wphase' = wreap$
$store' = store$
$displayed' = displayed$
$dirty' = dirty$
$batch' = batch$
$cleared' = cleared$
$snapScene' = snapScene$
$snapVal' = snapVal$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$readLock' = readLock$
$display' = display$
$shutting' = shutting$

On a dead peer (`OSError`) there is nothing to kill; the worker closes the dead descriptor and reconnects.

SendDead

$\Delta Repl$

$wphase = wsend$
$display = ddead$
$clientLock' = free$
$wphase' = wreconn$
$store' = store$
$displayed' = displayed$
$dirty' = dirty$
$batch' = batch$
$cleared' = cleared$
$snapScene' = snapScene$
$snapVal' = snapVal$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$readLock' = readLock$
$display' = display$
$shutting' = shutting$

8 Reap, Respawn, Reconnect

A stuck display still answers new connections, so it must be killed before a fresh one is started. `Reap` terminates the wedged process; `Ensure` starts a fresh, empty one; `Remark` re-marks every live scene so the fresh display is repainted from the store's current state. `Ensure` is enabled only from `ddead`: a fresh display is never spawned while one is alive, which is I5.

Reap

$\Delta Repl$

$wphase = wreap$
$display' = ddead$
$wphase' = wensure$
$store' = store$
$displayed' = displayed$
$dirty' = dirty$
$batch' = batch$
$cleared' = cleared$
$snapScene' = snapScene$
$snapVal' = snapVal$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$readLock' = readLock$
$clientLock' = clientLock$
$shutting' = shutting$

Ensure

$\Delta Repl$

$wphase = wensure$
$display = ddead$
$display' = dup$
$displayed' = (\lambda\, s : SCENE \bullet vempty)$
$snapScene' = \varnothing$
$snapVal' = vempty$
$wphase' = wremark$
$store' = store$
$dirty' = dirty$
$batch' = batch$
$cleared' = cleared$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$readLock' = readLock$
$clientLock' = clientLock$
$shutting' = shutting$

Remark

$\Delta Repl$

$wphase = wremark$
$dirty' = dirty \cup \{s : SCENE \mid store\, s = vlive\}$
$batch' = \varnothing$
$wphase' = wproc$
$store' = store$
$displayed' = displayed$
$cleared' = cleared$
$snapScene' = snapScene$
$snapVal' = snapVal$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$readLock' = readLock$
$clientLock' = clientLock$
$display' = display$
$shutting' = shutting$

`Reconn` handles the dead-peer path. A dead peer may come back as a fresh, empty display on reconnect, so this path is modelled honestly: the reconnected display shows nothing, and the worker re-marks every live scene — exactly as `Ensure/Remark` do after a reap. There is no process to kill, so `Reconn` reconnects, resets the display to empty, and re-marks in one step. I4 holds *because of* that re-mark: the fresh display starts blank, so the system cannot come to rest until every live scene is pushed again.

Reconn

$\Delta Repl$

$wphase = wreconn$
$display' = dup$
$displayed' = (\lambda\, s : SCENE \bullet vempty)$
$dirty' = dirty \cup \{s : SCENE \mid store\, s = vlive\}$
$batch' = \varnothing$
$snapScene' = \varnothing$
$snapVal' = vempty$
$wphase' = wproc$
$store' = store$
$cleared' = cleared$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$readLock' = readLock$
$clientLock' = clientLock$
$shutting' = shutting$

9 Ending a Cycle, Clear, Shutdown

A cleared cycle blanks the display *before* it repaints the batch. `ApplyClear` pushes the blank first, and `Snap` is guarded by `cleared = clear`, so no scene is painted until the clear has been applied. The worker then repaints each batch scene from the store's current value, so a `show` that coalesced after the `clear` survives.

This is a deliberate departure from the design text, which pushes the clear *after* the batch (`for scene in batch: push(...); if was_cleared: push_clear()`). Model-checking I4 against that order finds a lost update: a `clear` followed by a `show` within the worker's 16 ms coalescing window paints the new scene and then blanks it (Section 11). The clear-first order specified here is convergent for both `show`-then-`clear` and `clear`-then-`show`; the design document should be amended to match. The clear's own send is modelled as an immediate blank; its send has the same time limit and the same two failure outcomes as a scene send, which the scene-send operations already cover.

ApplyClear

$\Delta Repl$

$wphase = wproc$
$cleared = set$
$displayed' = (\lambda\, s : SCENE \bullet vempty)$
$cleared' = clear$
$store' = store$
$dirty' = dirty$
$batch' = batch$
$snapScene' = snapScene$
$snapVal' = snapVal$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$readLock' = readLock$
$clientLock' = clientLock$
$display' = display$
$shutting' = shutting$
$wphase' = wphase$

When the batch is empty and the clear (if any) has been applied, the worker goes idle.

ProcDone

$\Delta Repl$

$wphase = wproc$
$batch = \varnothing$
$cleared = clear$
$wphase' = widle$
$store' = store$
$displayed' = displayed$
$dirty' = dirty$
$batch' = batch$
$cleared' = cleared$
$snapScene' = snapScene$
$snapVal' = snapVal$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$readLock' = readLock$
$clientLock' = clientLock$
$display' = display$
$shutting' = shutting$

The display fails independently of the worker: an environment step may wedge it or kill it. These are the events the worker's next send detects.

DisplayWedge

$\Delta Repl$

$display = dup$
$display' = dstuck$
$store' = store$
$displayed' = displayed$
$dirty' = dirty$
$batch' = batch$
$cleared' = cleared$
$snapScene' = snapScene$
$snapVal' = snapVal$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$readLock' = readLock$
$clientLock' = clientLock$
$shutting' = shutting$
$wphase' = wphase$

DisplayCrash

$\Delta Repl$

$display = dup$
$display' = ddead$
$store' = store$
$displayed' = displayed$
$dirty' = dirty$
$batch' = batch$
$cleared' = cleared$
$snapScene' = snapScene$
$snapVal' = snapVal$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$readLock' = readLock$
$clientLock' = clientLock$
$shutting' = shutting$
$wphase' = wphase$

At clean shutdown the worker makes one last bounded flush of whatever is still dirty and then stops. The flush is best-effort: losing the last frame is acceptable because the process is exiting. `Restart` re-enters a fresh idle state, modelling luxd starting again, and keeps the model live so the deadlock check reports only genuine lock deadlocks, never this deliberate terminal stop.

ShutdownReq

$\Delta Repl$

$shutting = clear$
$shutting' = set$
$store' = store$
$displayed' = displayed$
$dirty' = dirty$
$batch' = batch$
$cleared' = cleared$
$snapScene' = snapScene$
$snapVal' = snapVal$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$readLock' = readLock$
$clientLock' = clientLock$
$display' = display$
$wphase' = wphase$

Flush

$\Delta Repl$

$wphase = widle$
$shutting = set$
$dirty' = \emptyset$
$wphase' = wstopped$
$store' = store$
$displayed' = displayed$
$batch' = batch$
$cleared' = cleared$
$snapScene' = snapScene$
$snapVal' = snapVal$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$readLock' = readLock$
$clientLock' = clientLock$
$display' = display$
$shutting' = shutting$

Restart

$\Delta Repl$

$wphase = wstopped$
$store' = (\lambda\, s : SCENE \bullet vempty)$
$displayed' = (\lambda\, s : SCENE \bullet vempty)$
$dirty' = \emptyset$
$batch' = \emptyset$
$cleared' = clear$
$snapScene' = \emptyset$
$snapVal' = vempty$
$storeLockMut' = \emptyset$
$mutScene' = \emptyset$
$readLock' = free$
$clientLock' = free$
$display' = dup$
$shutting' = clear$
$wphase' = widle$

10 Invariants

Five properties must hold across all reachable states. None is placed in the `Repl` schema: a predicate placed there would be enforced as a guard on every successor, silently disabling any operation that would break it and thereby masking the very defect this model exists to catch.

Invariant 1 — single writer. Only the worker ever holds the client send lock, and it holds it only while sending:

$$clientLock = held \Rightarrow wphase = wsend.$$

No mutator operation mentions `clientLock` in its effect, so no mutator can ever hold it. The design removes the two second-writer paths the current code has — the inline resend in the click dispatch and the beads app's direct render — so that the worker's `wsend` is the only place a send happens.

Invariant 2 — agent-path liveness. No mutator operation is guarded by `clientLock`, `display`, `readLock`, or `wphase`: `ReplaceBegin`, `ReplaceCommit`, and `ClearStore` depend only on the store write lock. So no state of the display or of the worker's send can disable a mutator. The witness (Section 11) is a reachable state in which the worker is stuck sending to a wedged display while a mutator holds the write lock and is free to commit.

Invariant 3 — no lock cycle. No thread holds the store lock and the client send lock at once:

$$\neg\ (readLock = held \land clientLock = held).$$

The worker takes the read lock in `Snap`, releases it in `SendBegin` *before* taking the client lock, and no mutator ever takes the client lock. The two locks are therefore always acquired in one order and never held together, so no cyclic wait can form.

Invariant 4 — no lost update. Whenever the system is at rest with the display up, the display shows the store's latest value for every scene. "At rest" means the worker is idle, nothing is dirty or cleared, no mutator holds the write lock, and no shutdown is in progress:

$$Quiescent \Rightarrow (\forall\, s : SCENE \bullet displayed\ s = store\ s),$$

where

$$\begin{aligned} Quiescent \ \widehat{=}\ & wphase = widle \land dirty = \varnothing \land cleared = clear \\ & \land\ storeLockMut = \varnothing \land shutting = clear \land display = dup. \end{aligned}$$

After a reap and respawn the fresh display is empty, so `Remark` re-marks every live scene; the system cannot come to rest until those scenes are pushed, which restores `displayed = store`.

Invariant 5 — one display. A fresh display is spawned only after the old one is killed: `Ensure` is enabled only from `display = ddead`, and `Reap` is the only step that reaches `ddead` on the recovery path. So the worker never holds two live displays.

11 Verification

Type-checking is a `fuzz` pass:

```
fuzz docs/hub_replicator.tex
```

I3, I4, and I5 are state properties, checked by asking ProB whether their negation is *reachable*. A goal reported *not found* means the invariant holds on every reachable state:

```
# Invariant 3 (must report: goal NOT found)
probcli docs/hub_replicator.tex -model_check -nodead \
  -goal "readLock = held & clientLock = held" \
  -p DEFAULT_SETSIZE 2
```

```
# Invariant 4 (must report: goal NOT found)
probcli docs/hub_replicator.tex -model_check -nodead \
  -goal "wphase = widle & dirty = {} & cleared = clear &
         storeLockMut = {} & shutting = clear & display = dup &
         card({s | s : SCENE & displayed(s) /= store(s)}) > 0" \
  -p DEFAULT_SETSIZE 2

# Invariant 5 (must report: goal NOT found)
probcli docs/hub_replicator.tex -model_check -nodead \
  -goal "wphase = wensure & display /= ddead" \
  -p DEFAULT_SETSIZE 2
```

I1 is corroborated by a reachability goal that must be *not found* — no state holds the client lock outside a send:

```
# Invariant 1 (must report: goal NOT found)
probcli docs/hub_replicator.tex -model_check -nodead \
  -goal "clientLock = held & wphase /= wsend" \
  -p DEFAULT_SETSIZE 2
```

I2 is a liveness witness that must be *found*: a reachable state in which the worker is sending to a wedged display while a mutator holds the write lock and can still commit — the agent path is not blocked by display I/O:

```
# Invariant 2 (must report: goal FOUND)
probcli docs/hub_replicator.tex -model_check -nodead \
  -goal "display = dstuck & clientLock = held & storeLockMut /= {}" \
  -p DEFAULT_SETSIZE 2
```

Deadlock freedom is the full-state-space check. `Restart` keeps the model live, so any deadlock reported would be a genuine lock deadlock:

```
# Deadlock freedom (must report: no deadlock / no counterexample)
probcli docs/hub_replicator.tex -model_check -p DEFAULT_SETSIZE 2
```

Finding — clear ordering. Model-checking I4 against the design document's stated order — push the batch scenes, then `if was_cleared: push_clear()` — returns *goal found*. Replace `ApplyClear` (which blanks first) with a `PushClear` that blanks at the end of the cycle, and this trace is reachable:

```
ClearStore              ; store emptied, cleared set
ReplaceBegin/Commit s1  ; a show() lands after the clear, within
                        ;   the 16ms coalescing window; store s1 = vlive
Drain                   ; batch = {s1}, cleared carried
Snap; SendBegin; SendOk ; display now shows s1 = vlive
PushClear               ; display blanked to empty
-- rest: displayed s1 = vempty, store s1 = vlive  (lost update)
```

A `clear` immediately followed by a `show` is an ordinary sequence, so this is a reachable defect, not a corner case. The clear-first order in `ApplyClear` closes it: with clear-first the goal returns *not found*. The design document should be amended to blank before repainting.

Note — the dead-peer path. Both send-failure paths re-mark every live scene. A dead peer may return as a fresh, empty display on reconnect, so `Reconn` resets the display to empty and re-marks every live scene, exactly as the reap path does; it differs only in having no process to kill. I4 holds on this path *because of* the re-mark: were `Reconn` to reset `displayed` without re-marking — re-queuing only the scene in flight — the `Quiescent` violation would be reachable for the scenes that were already clean, since the fresh display shows none of them. The re-mark is what closes that gap.

12 Fidelity: reproducing the store race

A model that cannot reproduce the known defect proves nothing. The buggy variant is this specification with one change: the store's reader-writer exclusion is removed from the worker's snapshot. Concretely, delete the guard `storeLockMut = ∅` from `Snap`, so the worker can copy a scene's value while a mutator is inside `replace_scene` and the scene is `vtorn`. The torn read then becomes reachable:

1. Mutator m runs `ReplaceBegin` on scene s: it takes the write lock and sets `store` s `= vtorn`, with `storeLockMut = {m}`.
2. Meanwhile the worker has a batch containing s and is at `wproc`. Without the exclusion guard, `Snap` fires: it copies `snapVal = store` s `= vtorn`.
3. The worker is now about to send a half-updated scene: a state with `snapVal = vtorn`.

The property that separates the two designs is therefore

```
# must be NOT found for the correct spec, FOUND for the buggy variant
probcli <spec>.tex -model_check -nodead \
  -goal "snapVal = vtorn" -p DEFAULT_SETSIZE 2
```

Under the intact exclusion the worker's `Snap` cannot fire while any mutator holds the write lock, so `snapVal` is never `vtorn`; the reachability search returns *goal not found*. With the guard removed it returns *goal found* — the trace above. That difference is the evidence that the model detects the race the store lock was added to prevent.